

Tarea 2

Mariano Fernández

June 25, 2026

Contents

1	Introducción	2
2	Dataset y representación numérica de texto	2
2.1	Preparación del dataset	2
2.2	Balance de clases	2
2.3	Representación Bag of Words	3
2.4	Representación TF-IDF	3
2.5	PCA sobre TF-IDF	4
3	Entrenamiento y evaluación de modelos	5
3.1	Multinomial Naive Bayes	5
3.2	Validación cruzada y búsqueda de hiperparámetros	6
3.3	Entrenamiento con mejores hiperparámetros	7
3.4	Random Forest	9
3.5	Cambio de medio de prensa	10
3.6	Extracción de features alternativa	10
3.7	Clasificación a nivel de título	12

1 Introducción

Este informe presenta los resultados y el análisis correspondientes a la Tarea 2 del curso *Introducción a la Ciencia de Datos*. Esta tarea es la continuación de la Tarea 1, donde se utiliza el mismo dataset *All the News 2.1 – Component One*. El dataset contiene artículos en inglés de distintos medios de prensa, con información como el título, cuerpo del artículo, fecha de publicación y medio de prensa.

2 Dataset y representación numérica de texto

2.1 Preparación del dataset

Para este estudio se utiliza un subconjunto del dataset original, formado por las publicaciones de los tres medios de prensa más representados: *Reuters*, *The New York Times* y *CNBC*. Luego se aplicó la función `clean_text` (visto en la Tarea 1) sobre la columna del cuerpo del artículo con el objetivo de estandarizar el texto.

Los conjuntos de Train y Test se generaron utilizando una partición estratificada, con un tamaño de test del 30% y una semilla de 42 para garantizar la reproducibilidad. El resultado es de 10425 artículos para entrenamiento y 4469 para prueba, manteniendo la proporción de las clases en ambos conjuntos.

2.2 Balance de clases

En la figura 1 se muestra la distribución de clases del dataset, separando en el conjunto de Train y el conjunto de Test. Se observa que la proporción se mantiene en aproximadamente igual para ambos conjuntos con un 63% de artículos de *Reuters*, 19% de *The New York Times* y 18% de *CNBC*. Esto también muestra un desbalance claro donde la mayoría de los artículos pertenecen a *Reuters*.

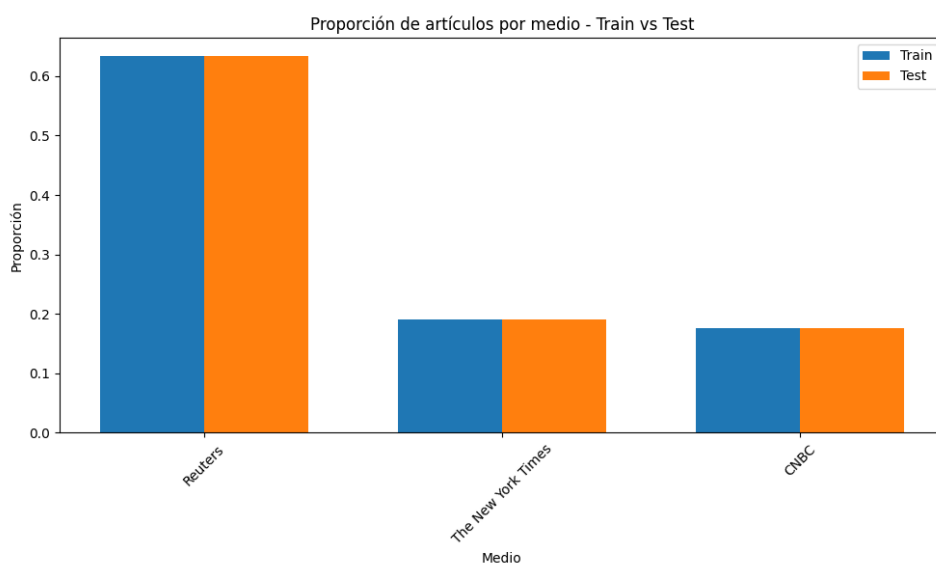


Figure 1: Distribución de clases

2.3 Representación Bag of Words

Para poder entrenar un modelo de clasificación sobre datos de texto es necesario transformarlos a una representación numérica que pueda ser procesada por dichos algoritmos. Un posible método es *Bag of Words* (BoW), que consiste en primero definir el vocabulario del conjunto de entrenamiento, es decir, todas las palabras únicas que aparecen en él. Luego, cada artículo se representa como un vector de longitud igual al tamaño del vocabulario, donde cada posición indica la cantidad de veces que la palabra correspondiente aparece en el texto. El resultado es una matriz de $N \times M$, donde N es el número de artículos y M es el tamaño del vocabulario.

Por ejemplo, si el conjunto de datos es:

```
"london reuters the world"
"london aug 8 reuters"
```

El vocabulario sería: ["london", "reuters", "the", "world", "aug", "8"] y la representación BoW de cada artículo sería

```
[1, 1, 1, 1, 0, 0]
[1, 1, 0, 0, 1, 1]
```

En el caso particular del dataset utilizado, el tamaño del vocabulario es de 87044 palabras únicas, lo que resulta en una matriz de 10425×87044 para el conjunto de entrenamiento y 4469×87044 para el conjunto de prueba. Si la matriz de entrenamiento se almacenara en un array sin ninguna optimización, ocuparía $10425 \times 87044 \times 4$ bytes (asumiendo enteros de 32 bits) lo que equivale aproximadamente a 3.4 GB de memoria. Dado que para un artículo en particular solo van a aparecer muy pocas palabras del vocabulario, la mayoría de los campos de la matriz serán cero y el resultado es una matriz muy dispersa. Scikit-learn utiliza una representación de matriz dispersa que solo almacena los valores no nulos, lo que reduce significativamente el uso de memoria. En este caso la matriz de entrenamiento solo ocupa aproximadamente 25.30 MB de memoria.

2.4 Representación TF-IDF

Simplemente usar BoW como representación del texto puede llevar a que se pierda mucha información, una desventaja es que solo se cuenta la cantidad de veces que aparece una palabra pero no se tiene en cuenta el orden o la relación con palabras vecinas. Como forma de capturar parte de esta información se suele utilizar lo que se conoce como N-gramas, que son secuencias de N palabras consecutivas. Por ejemplo si se utiliza $N=2$, se van a contar no solo las palabras individuales sino también las secuencias de dos palabras consecutivas, logrando así capturar contexto local. Esto tiene un efecto importante en el tamaño del vocabulario, ya que cada par de palabras va a representar una nueva feature, lo que hace que incremente el tamaño de la matriz de features y el uso de memoria.

Otro punto débil de la representación BoW es que el conteo absoluto de palabras hace que el largo del texto tenga un impacto grande en la representación, cuando lo relevante es el contenido o tema del artículo. A esto se suma que las palabras más representadas van a ser las palabras más comunes en el idioma, y seguramente no aporten mucha información relevante para la tarea

de clasificación. Para solucionar esto se suele utilizar la representación *Term Frequency - Inverse Document Frequency* (TF-IDF), que consiste en dos etapas: primero normalizar el conteo de palabras por el largo del texto (TF) y segundo multiplicar por un factor que penaliza las palabras que aparecen en muchos artículos (IDF).

- $TF = \frac{\text{count}}{\text{total}}$
- $IDF = \log \frac{N}{n}$, donde N es la cantidad de artículos y n la cantidad de artículos donde aparece la palabra.
- $TF-IDF = TF \times IDF$

En este estudio se utiliza la implementación de la librería Scikit-learn, que es una variante de la definición anterior. En particular se agrega un suavizado a la definición de IDF para evitar valores extremos para palabras muy poco comunes y que términos presentes en todos los documentos no tengan multiplicador nulo:

- $IDF = \log \frac{1+N}{1+n} + 1$

Además en lugar de normalizar TF por el total de palabras, se utiliza la norma L2 sobre el vector resultante, de forma de que todos los vectores tengan norma 1.

2.5 PCA sobre TF-IDF

Una forma de visualizar la representación TF-IDF es utilizando PCA para reducir la dimensionalidad de los datos a 2 dimensiones. En la figura 2 se muestra el resultado de aplicar PCA sobre la matriz TF-IDF del conjunto de entrenamiento, cada punto representa un artículo y su color el medio de prensa al que pertenece. Se puede observar que en su mayoría los artículos de *Reuters* se pueden separar bastante bien de los de *The New York Times* pero *Reuters* y *CNBC* tienen un overlap muy grande, y por lo menos en esta visualización no son separables. De todos modos la representación de PCA es una simplificación grande del problema y es posible que viéndolo en un espacio de mayor dimensión los datos sí sean separables. Como argumento a favor de este punto, en la figura 3 se muestra la varianza explicada acumulativa según la cantidad de componentes que se toman, y las primeras dos componentes solo representan aproximadamente el 4% de la varianza total.

También se estudió cómo variaba la representación de PCA modificando el preprocesamiento de los datos, los resultados se pueden ver en la figura 4. La forma de los clusters tiene algunas variaciones pero el patrón general se mantiene, en particular en todos los casos los artículos de *Reuters* parecen ser separables en dos conjuntos diferentes, uno bastante alejado al resto de los medios y fácil de separar y otro que en esta proyección es indistinguible de *CNBC*. Lo primero puede explicarse porque muchos artículos de *Reuters* contienen plantillas que facilitan la separación, como son los artículos de tipo *BRIEF*, además la separación parece estar más marcada cuando no se usa IDF probablemente por el peso del largo de los artículos. Por otro lado, una posible explicación del overlap entre *Reuters* y *CNBC* que también se observa en todos los casos puede ser lo visto en la Tarea 1, donde se concluyó que muchos artículos de *CNBC*

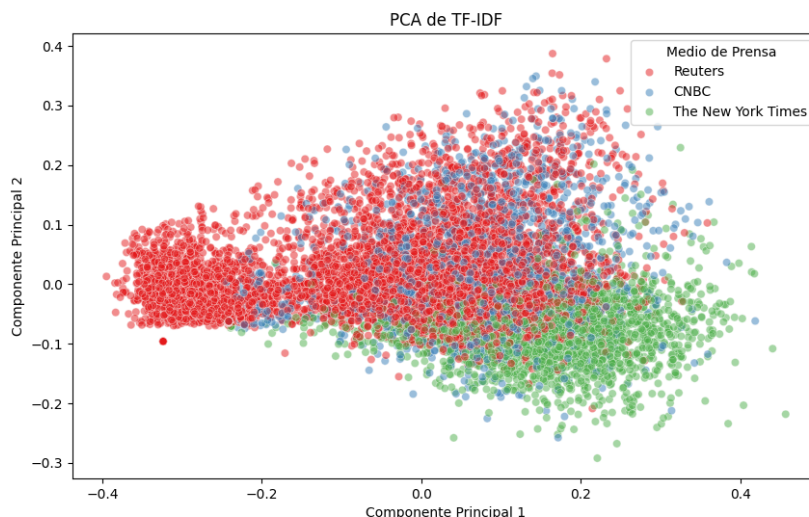


Figure 2: PCA sobre TF-IDF

son realmente artículos de *Reuters*. Para confirmar estas hipótesis sería interesante agregar los filtros propuestos en el informe de la Tarea 1 y ver si los resultados de PCA cambian.

Comparando las figuras 4a y 4b se puede ver que filtrar las stop words parece tener un efecto positivo en la separación de los clusters, esto tiene sentido si se considera que aplicar este filtro resulta en que los artículos estén representados por palabras más relevantes y diferentes entre medios, lo que resulta en features más separables. Por otro lado agregar bigramas (figura 4c) no parece tener un gran efecto en la separación de los clusters aunque es importante notar que usar n-gramas aumenta la dimensionalidad del problema mientras que PCA lo reduce a las 2 componentes principales. Como ya se vio, quitar la ponderación IDF (figura 4d) parece dejar un subconjunto de artículos de *Reuters* muy separado del resto, pero parece empeorar la separación de los otros clusters.

3 Entrenamiento y evaluación de modelos

3.1 Multinomial Naive Bayes

A partir de la representación TF-IDF se entrena un modelo Multinomial Naive Bayes en el conjunto de entrenamiento, con los parámetros predeterminados y se evalúa en el conjunto de prueba donde se obtiene un accuracy del 70.13%. Esto quiere decir que el modelo es capaz de clasificar correctamente el 70% de los artículos del conjunto de test, el problema es que este número puede ocultar problemas con el modelo, por ejemplo como se vio anteriormente *Reuters* predomina en el dataset por lo que también va a tener un peso más grande en el cálculo del accuracy. Para tener una mejor visión del desempeño del modelo, en la figura 5 se muestra la matriz de confusión en el conjunto de prueba que se acompaña con la tabla de métricas 1, como era de esperarse el modelo tiende a clasificar un artículo como perteneciente a *Reuters*, por esto su Recall en *Reuters* es casi 1, solo 3/2830 se clasifican erróneamente como otra clase. El problema aparece al ver que la Precision de la clase *Reuters* es de 68%, es decir que de todos los

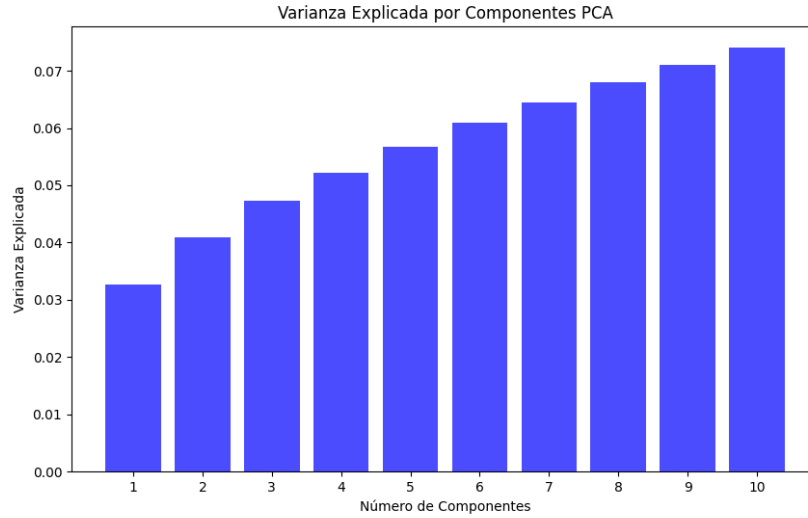


Figure 3: Varianza explicada en PCA por cantidad de componentes

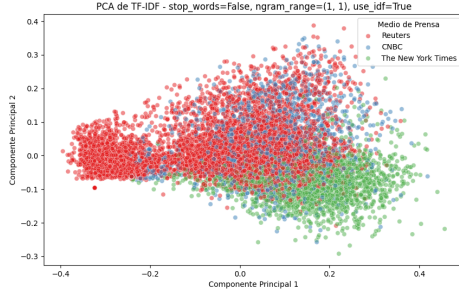
artículos que se predicen como *Reuters* 32% son de otra clase. El efecto es más claro cuando se estudian las métricas de las otras clases, principalmente *CNBC* que tiene una precisión de 100% pero un recall de 1% eso quiere decir que todos los artículos que predice como *CNBC* los predice correctamente pero son solo 8, que representan el 1% de los textos del medio, la gran mayoría de los artículos de *CNBC* se clasifican como *Reuters*. Esto coincide con la poca separabilidad entre *Reuters* y *CNBC* que se vio en la visualización de PCA 2.

Métrica	Reuters	The New York Times	CNBC
Precision	0.6812	0.9614	1.0000
Recall	0.9989	0.3509	0.0102
F1-score	0.8100	0.5142	0.0201

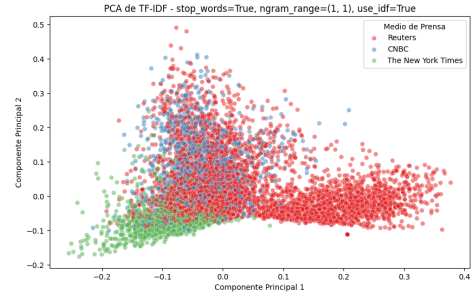
Table 1: Resultados del modelo Multinomial Naive Bayes base

3.2 Validación cruzada y búsqueda de hiperparámetros

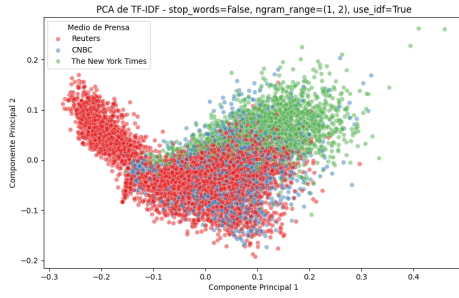
Hasta el momento se entrenó solo un modelo sin modificar los parámetros predeterminados pero una parte importante del entrenamiento es la búsqueda de hiperparámetros del modelo, que pueden afectar considerablemente el desempeño final. Una consideración clave a tener es que no se puede usar el mismo conjunto de test para esta búsqueda de hiperparámetros porque podría pasar que estemos sobreajustando a ese conjunto pero no represente la realidad del problema. Para solucionar este problema se suelen usar dos estrategias: En lugar de solo particionar el conjunto en entrenamiento y prueba, se vuelve a particionar el conjunto de entrenamiento para obtener el conjunto de validación, este es similar al conjunto de Test que se usa para comparar diferentes configuraciones del modelo (hiperparámetros) y luego de obtener la mejor se evalúa en el conjunto de prueba para obtener una estimación del desempeño del modelo. El problema con esa estrategia es que se reduce mucho la cantidad de datos de entrenamiento y los mejores hiperparámetros elegidos dependen mucho del split realizado, por esto otra estrategia muy



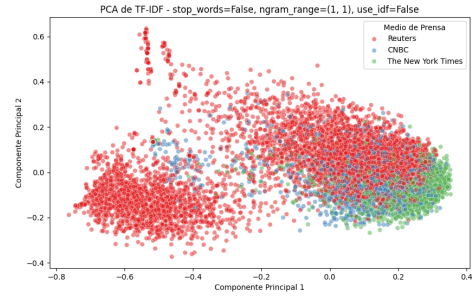
(a) Base (sin filtrar stop words, unigramas, IDF=True)



(b) Filtrando stop words, unigramas, IDF=True



(c) Sin filtrar stop words, con bigramas (ngram_range=(1,2)), IDF=True



(d) Sin filtrar stop words, unigramas, sin ponderación IDF (use_idf=False)

Figure 4: Comparación de PCA sobre distintas configuraciones del vectorizador de texto.

utilizada es validación cruzada donde se repite N veces la estrategia anterior. Se generan N splits, de forma de los N conjuntos de validación resultante no tengan overlap y la suma de todos cubra todo el conjunto de train, luego para cada configuración se repite el entrenamiento en cada uno de los N splits y se reportan los resultados finales en cada partición. Como paso final se elige la configuración que obtuvo el mejor desempeño promedio en los N splits y se entrena un modelo final con esa configuración utilizando todo el conjunto de entrenamiento.

En la figura 6 se muestra el resultado de la búsqueda de hiperparámetros realizada para el modelo Multinomial Naive Bayes, para todas las combinaciones de los siguientes hiperparámetros:

```
alpha: [0.1, 0.25, 0.5, 0.75, 1.0]
fit_prior: [True, False]
```

Obteniendo un total de 10 configuraciones diferentes. De la gráfica 6 se concluye que $\alpha=0.1$ suele obtener mejores resultados y para ese caso no hay una gran diferencia entre si usar fit_prior o no.

3.3 Entrenamiento con mejores hiperparámetros

Los mejores hiperparámetros obtenidos de la sección anterior son: $\alpha=0.1$ y $\text{fit_prior}=\text{True}$, se usan estos parámetros para entrenar el modelo en todo el conjunto de train obteniendo un accuracy de 80.47% en el conjunto de Test, lo que significa una mejora de +10 puntos sobre usar los hiperparámetros predeterminados. Además viendo la matriz de confusión de la figura 7

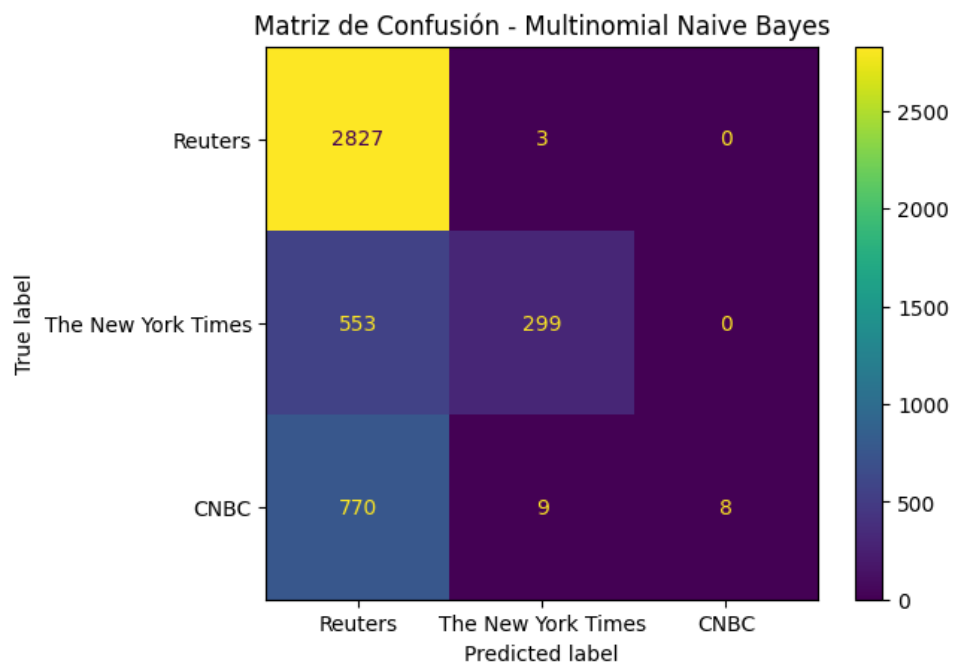


Figure 5: Matriz de confusión del modelo Naive Bayes

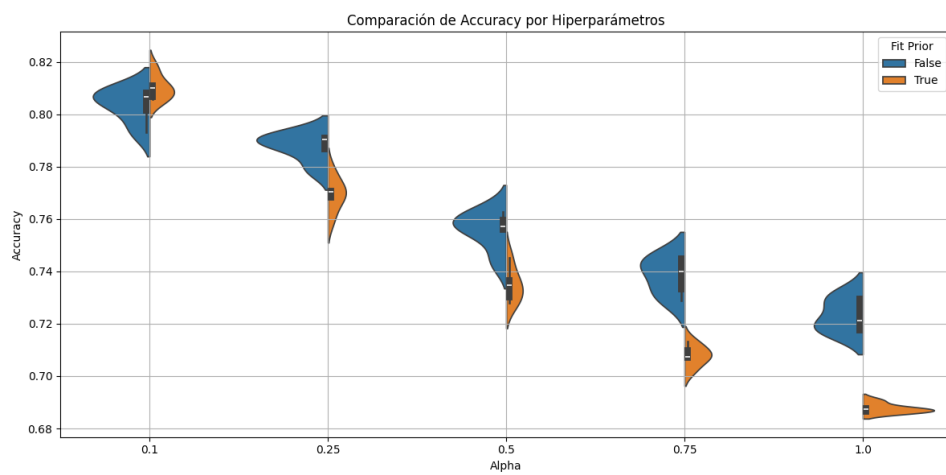


Figure 6: Resultados de GridSearchCV

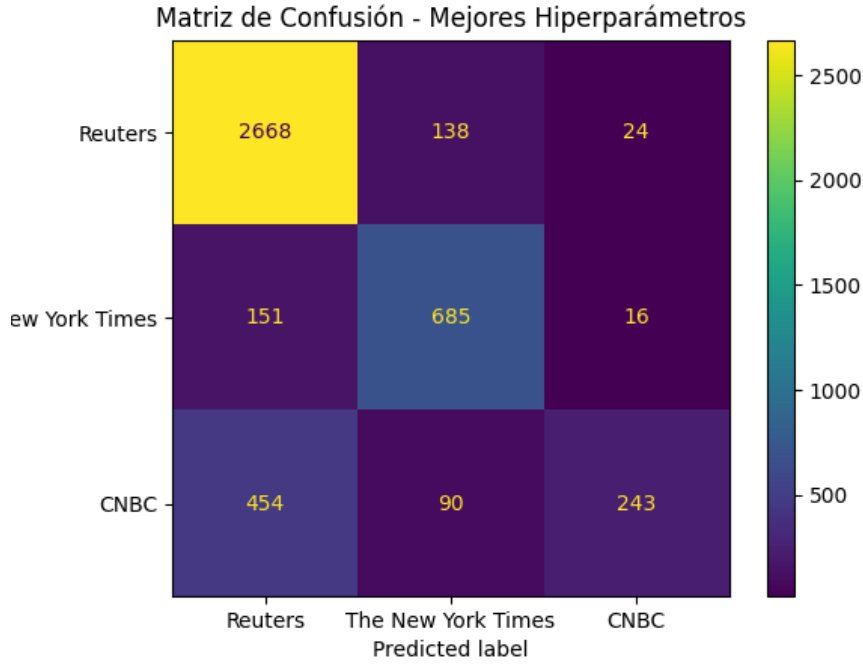


Figure 7: Matriz de confusión del modelo con mejores hiperparámetros

y las métricas de Precision y Recall por medio de la tabla 2 este modelo ya no presenta tanto el problema de clasificar todos los artículos como *Reuters*. El problema que se mantiene es el Recall de *CNBC* aún es muy bajo, 30%, lo que refuerza la hipótesis que muchos artículos de *CNBC* no son separables de los de *Reuters*.

Métrica	Reuters	The New York Times	CNBC
Precision	0.8152	0.7503	0.8587
Recall	0.9428	0.8040	0.3088
F1-score	0.8743	0.7762	0.4542

Table 2: Resultados del modelo con mejores hiperparámetros

3.4 Random Forest

Como modelo alternativo a Multinomial Naive Bayes se decidió entrenar un modelo Random Forest, que es un modelo de clasificación basado en árboles de decisión. En resumen, en lugar de entrenar un solo árbol de decisión, se entrenan varios árboles, haciendo modificaciones aleatorias en los datos de entrenamiento de cada uno y en las features que se utilizan para cada árbol para asegurar que cada árbol sea diferente. Luego al momento de clasificar un artículo se hace una votación para obtener la predicción final. El uso de Random Forest en lugar de un solo árbol de decisión tiene varias ventajas, principalmente suele generalizar mejor ya que un problema de los árboles de decisión es que tienden a sobreajustar a los datos de entrenamiento, una pequeña modificación en los datos de entrenamiento puede causar grandes cambios en el modelo. El modelo se entrenó con los hiperparámetros predeterminados, obteniendo un accuracy de 83.11% en el conjunto de prueba, este resultado ya es mejor que el mejor modelo Naive Bayes con

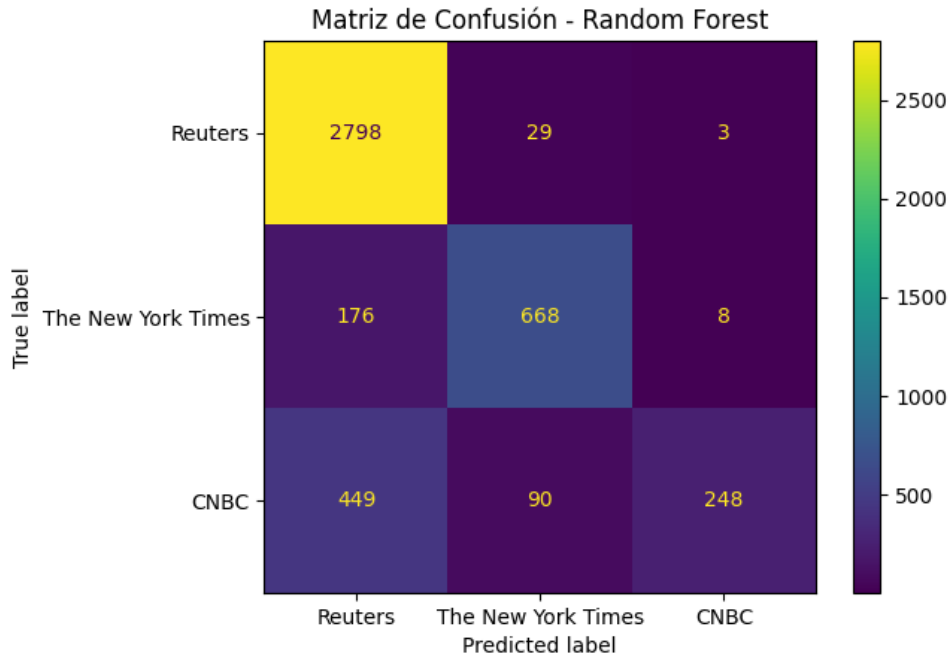


Figure 8: Matriz de confusión del modelo Random Forest

hiperparámetros optimizados. Cuando se estudia la matriz de confusión se ve un patrón similar al modelo anterior, siendo *Reuters* la clase con mejor desempeño, luego *The New York Times* y por último *CNBC* con la mayoría de los artículos clasificados como *Reuters*.

3.5 Cambio de medio de prensa

Como gran parte de los problemas vistos en este informe surgen por el desbalance que presentan la cantidad de artículos de *Reuters* o un problema en los datos que causa que *CNBC* no sea separable de *Reuters*, se optó por un subconjunto alternativo. Se seleccionaron los top 3 medios de prensa del dataset, descartando *Reuters*, obteniendo los siguientes medios: *The New York Times*, *CNBC* y *The Hill*. En la figura 9 se puede observar como la distribución de clases es mucho más pareja que en el conjunto original representado por la figura 1.

El nuevo conjunto de datos parece definir una tarea más fácil para el modelo, una posible razón es que al eliminar *Reuters* se reduce bastante el ruido en los datos. El modelo Multinomial Naive Bayes, entrenado con los hiperparámetros predeterminados obtuvo un accuracy de 87% en el conjunto de prueba. Además en la matriz de confusión 10 se puede ver un buen desempeño, sobre todo al separar *The New York Times* y *The Hill*, esto puede ser la diferencia temática de los medios. Por otro lado todavía se puede ver que gran parte de los artículos de *CNBC* se siguen clasificando erróneamente, en este caso la mayoría de los errores son porque el modelo predice *The New York Times*.

3.6 Extracción de features alternativa

La representación TF-IDF mantiene más información que utilizando BoW, pero aún así se pierde información importante del texto. Un método alternativo es utilizar un método que genere embeddings de palabras, estos suelen ser vectores de mucho menor dimensión que la

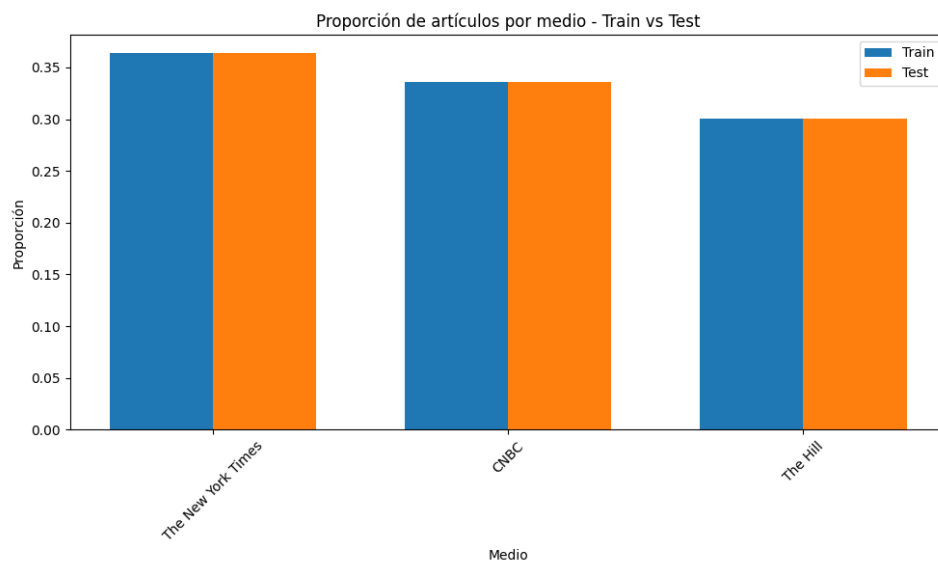


Figure 9: Distribución por clase para subconjunto alternativo.

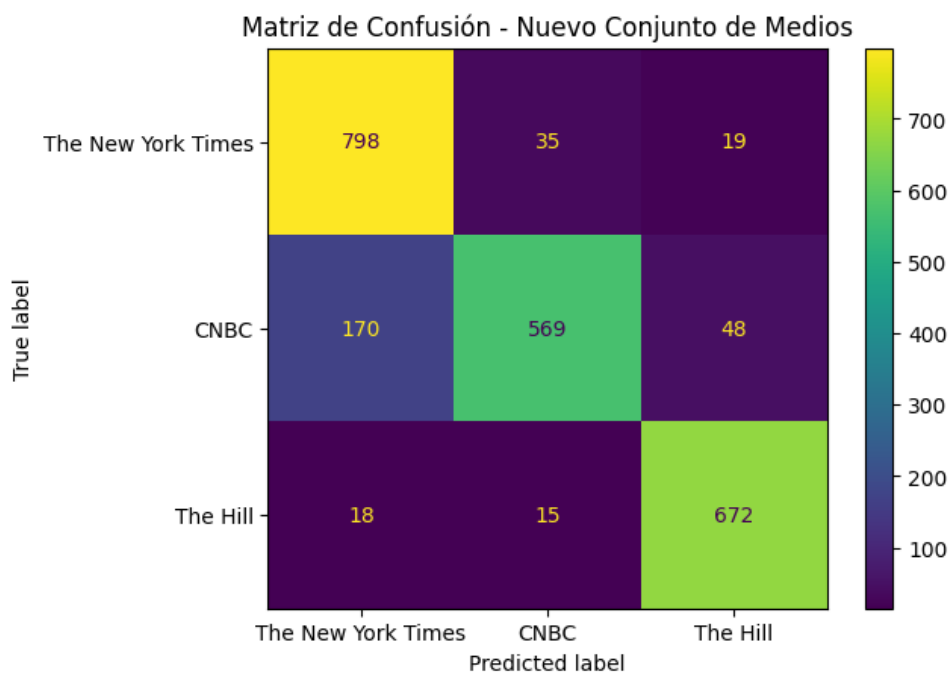


Figure 10: Matriz de confusión del modelo Naive Bayes con cambio de medio de prensa

representación TF-IDF y se entrenan para capturar relaciones semánticas entre palabras. Es decir dos embeddings de palabras que tengan un significado similar van a estar más cerca en el espacio de embeddings. Un algoritmo muy utilizado para generar embeddings en tareas de texto es Word2Vec, que se entrena con un corpus de texto y genera un embedding para cada palabra.

Aplicar Word2Vec sobre el dataset agrega una complejidad extra al problema, ya que ahora para cada artículo se tiene una secuencia de embeddings de palabras, en lugar de un vector de largo fijo. Para poder entrenar los mismos modelos de clasificación que antes, es necesario agregar una etapa que lleve estas secuencias de embeddings a un vector de largo fijo, una forma sería promediar todos los embeddings de palabras del artículo para obtener un único vector de features.

Otro enfoque que puede llegar a mejores resultados es utilizar un modelo que tome secuencias de largo variable como entrada, como por ejemplo una red neuronal recurrente (RNN) o un modelo transformer. Estos modelos son más complejos, requieren más tiempo de entrenamiento y más datos para generalizar bien, pero pueden hacer un mejor uso de toda la información del texto.

3.7 Clasificación a nivel de título

Otro experimento interesante es ver si es posible clasificar el artículo utilizando sólo el título, es de esperar un desempeño peor que usando todo el cuerpo del artículo ya que la cantidad de información es mucho menor. A pesar de esto entrenando un modelo Multinomial Naive Bayes, con los hiperparámetros predeterminados se obtuvo un 70% de accuracy en el conjunto de prueba. Esto se puede explicar por varias razones, en parte por el desbalance en las clases que ya se vio, por cómo se calcula la métrica de accuracy si el modelo tiende a predecir *Reuters* la métrica da mejor, esto además es respaldado por la matriz de confusión 11 donde se ve que la mayoría de los artículos son clasificados como *Reuters*.

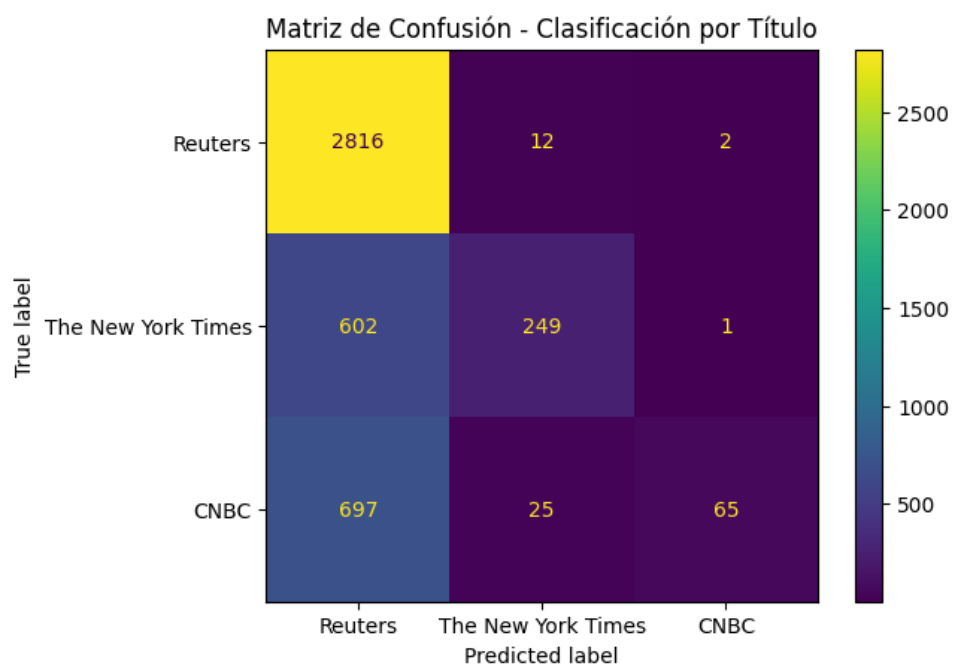


Figure 11: Matriz de confusión del modelo Naive Bayes con clasificación a nivel de título